

Bang-Bang and PID Control of a Motor-Flywheel System

Touchton, Noah

Section 12105 Date 10/25/25

Abstract— This experiment investigates closed-loop control of a DC motor–flywheel system using Bang-Bang and PID controllers to compare performance in position tracking and transient response. The system, previously modeled as a first-order process in Lab 1 [1], was controlled using a LabVIEW interface that implemented Bang-Bang, manually tuned PID, and Ackermann-tuned PID schemes. Each controller was tested under sinusoidal and square-wave position commands at 0.25 Hz. The Bang-Bang controller provided simple but oscillatory control, with RMS errors of 6–8° for sinusoidal input and 40–60° for square input. Increasing the proportional gain in the PID controller reduced steady-state error but increased overshoot, while manual tuning achieved the best overall tracking accuracy (RMS $\approx 5^\circ$). The Ackermann-tuned controller met the desired 0.3 s settling time and the 5 % overshoot target. Overall, manual PID tuning produced the most stable and accurate closed-loop performance.

Index Terms—Ackermann, Bang-Bang, Closed-Loop Feedback, PID

I. INTRODUCTION

THE objective of this lab is to implement and compare two different types of closed-loop controllers, bang-bang and PID, to control the rotation of a motor-platen system. The bang-bang controller is implemented with a deadband and a duty cycle. The PID controller will be manually tuned as well as tuned using desired overshoot and settling time specifications to solve for the control constants. The goal is to compare each controller's performance quantitatively to explain why differences occur based on control theory and system behavior.

As detailed in [1], the governing electrical and mechanical equations were linearized and reduced to a first order function shown in (1). The identified parameters in Lab 1 were used directly in the controller design pole placement calculations.

$$\frac{\Omega(s)}{V(s)} = \frac{K}{\tau s + 1} \quad (1)$$

Control Methods

Using control theory, controllers can provide an input signal into the system to achieve a desired output. The controller takes into account the error between the actual signal and the desired output to determine the input signal. Two types of controllers were used in this lab, Bang-Bang and PID.

Bang-Bang Controller

A bang-bang controller provides a fixed signal in the direction opposite of the error. The fixed signal is based on the duty cycle chosen for the controller. If the error is positive the controller will provide a negative signal until the error is no longer positive and vice versa. The deadband is implemented to prevent the controller from providing an input when the error is close to zero. When the error is within the designated deadband the controller will not provide an input. This prevents excessive oscillation, and the performance of the bang-bang controller depends on the chosen duty cycle and deadband.

PID Controller

A PID (Proportional-Integral-Derivative) controller provides continuous control by scaling input based on proportional, integral, and derivative terms. The proportional term works very similarly to the bang-bang controller but the magnitude of the input scales with the magnitude of the error. The integral term accumulates past error trends to eliminate long term bias. The derivative term attempts to predict future error using the rate of change of the error and attempts to counteract it. Tuning these three constants allows for a trade-off between speed, overshoot, and stability. These constants can be manually tuned as well as calculated from desired response parameters.

Ackermann Method

The Ackermann Method is a method for finding the PID gain constants using mathematical models. This method places the poles at specific, desired locations in the complex plane. This method uses a state-space representation described in [1] of the system to find a vector K that contains all the feedback gains.

To place the poles at these locations, desired system response properties must be set. In this lab, a desired 2% Settling time T_s and overshoot P.O. were set. These parameters can be used to find the desired dampening ratio ζ and the desired natural frequency ω_n as shown in (2-3) from [2].

$$\zeta = \frac{(\ln(\frac{P.O.}{100}))^2}{\pi^2 + (\ln(\frac{P.O.}{100}))^2} \quad (2)$$

$$\omega_n = \frac{4}{\zeta T_s} \quad (3)$$

Root Mean Square

Root Mean Square (RMS) is a statistical measure that quantifies the magnitude of a varying signal. The equation for how it is calculated is shown in (4) as described in [3]. In the context of control signals, the RMS represents the average deviation between the desired and actual response. A low RMS means there is low error in the control method.

$$X_{RMS} = \sqrt{\frac{1}{N} \sum_{i=1}^N (X_i)^2} \quad (4)$$

II. METHODS

Hardware

This lab uses the same motor-flywheel system and DAQ described in [1].

Software

A custom LabVIEW VI was developed to implement controllers. The VI contains a signal generator as described in [1]. The signal generator serves as the desired angular position, which when the measured angular position is subtracted from, creates an error signal. This signal is fed into the controller to create a feedback loop. The controller can be toggled between a bang-bang controller or a PID controller. The parameters of the controller can be toggled in real time to observe how the system responds.

A Python analysis script was used to automatically process all collected data. The script reads each dataset, standardizes the column names, and computes the error between desired and measured position when applicable. For square-wave tests, the script detects step transitions and computes rise time, settling time, and percent overshoot from each transient response. For sine-wave tests, it calculates root-mean-square (RMS) error and command effort over each cycle. The script then summarizes these metrics into comparison tables for each controller and waveform type, enabling quantitative performance evaluation.

Procedure

First, the desired amplitude was selected. This was done by adding 90 to the sum of the final four digits of your UFID. The turn on command was set to zero and the frequency was set to 0.25Hz. For this lab the amplitude was set to 99.

Next, the bang-bang controller was implemented with a chosen duty cycle and deadband. 6 trials were conducted with the deadband set to 0, 5, and 10 with the duty cycle set to 0.5 and 1.0. Each trial ran the square wave for about 4 cycles and recorded the data.

Then, the proportional part of the PID controller was implemented. 3 trials were conducted with the gain constant set

to 0.75, 2.0, and 5.0. Each trial ran the square wave for about 4 cycles and recorded the data.

After this, the previous two steps were repeated using a sinusoidal wave with the same frequency. This resulted in 9 additional trials with data collection.

Lastly, the PID was tuned before it was implemented. This was done using two methods. The first was adjusting the values manually until the response looked good. This was done by increasing the proportional gain constant until the system had a fast response but not so much that it had significant overshoot. Then, the derivative controller was increased until the overshoot was reduced. Finally, the integral controller was increased until oscillation was reduced. The values for these constants were recorded and both square and sine wave data was recorded for about 4 cycles each. The second method was calculating the desired gain constants for the proportional and derivative controllers using Ackermann's method and running the square wave with those constants. The data was collected for about 4 cycles.

Calculations

After all trials were collected, the recorded data was processed using a Python analysis script to quantify performance and effort for each controller configuration. The objective was to extract numerical metrics that could be compared across all controllers and waveforms. The python analysis script parsed through the data and separated the data by controller and waveform type. For each trial the code computed the mean, standard deviation, and RMS for the error and command effort. The code calculated the rise time, percent overshoot, and settling time for the square inputs.

To calculate the PID gain values using Ackermann's method, the DC gain and time constant calculated in Lab 1 through a first-order system approximation were used in the state-space calculation. These values establish the system dynamics such that the python script can solve for what PID constants place the poles at the desired locations.

Units and Scaling

To find the proper voltage the script must convert the command into voltage using the integer command factor as well as the voltage of the motor. The command will be from -1023 to 1023 so this scaling factor can be used to find the voltage. The LabVIEW software also outputs the rotational velocity in degrees per second so the script must convert this to radian per second.

III. RESULTS

Bang-Bang Response

For the bang-bang controller, both sine tracking and square wave responses were tested using different deadbands and duty cycles. The duty cycle was tested at 0.5 and 1.0 alongside the deadband being tested at 0, 5, and 10.

Table I shows the mean, standard deviation, and root mean square of the command signal and error signals for all of the bang-bang trials.

TABLE I
BANG-BANG CONTROLLER PERFORMANCE

Wave	d	Duty Cycle	Mean cmd	Std cmd	RMS cmd	Mean Err °	Std Err °	RMS (err °)
Sine	0	0.5	62.91	30.49	69.91	0.29	6.44	6.45
	5	0.5	63.02	30.45	69.99	-0.09	3.8	3.8
	10	0.5	63.01	30.42	69.97	1.51	7.72	7.87
Square	0	0.5	-0.38	99.01	99	1.13	61.98	61.99
	5	1	0.39	99.01	99	6.64	57.51	57.89
	10	1	-0.09	99.01	99	6.71	43.67	44.18

To further characterize the performance of the bang-bang controller, second order system metrics were analyzed for the square wave responses. Table II shows the rise time, percent overshoot, and settling time calculated for the square wave bang-bang trials.

TABLE II
BANG-BANG SQUARE SECOND ORDER SYSTEM METRICS

Deadband (degrees)	Duty Cycle	Rise Time (s)	Percent Overshoot (%)	Settling Time (s)
0	0.5	0.332	105.514	0.470
5	0.5	0.329	104.513	0.449
10	0.5	0.328	102.420	0.452
0	1.0	0.152	112.537	0.242
5	1.0	0.324	113.345	0.363
10	1.0	0.155	112.336	0.22

PID Response

For the PID response, the proportional gain constant was set to values of 0.75, 2, and 5 while the other two gain constants remained at 0.

Table III shows the mean, standard deviation, and root mean square of both the command and error signal for all the proportional trials.

TABLE III
PROPORTIONAL CONTROLLER PERFORMANCE

Wave	K _p	Mean cmd	Std cmd	RMS cmd	Mean err °	Std err °	RMS err °
Sine	0.75	63.26	30.22	70.1	24.73	30.2	39.05
	2	63.09	30.47	70.06	48.71	36.1	60.63
	5	62.99	30.56	70.01	12.2	28.5	31
Square	0.75	-0.03	99.01	99	-30.02	99	103.4
	2	0.1	99.01	99	2.94	71.3	71.3
	5	-0.26	99.01	99	3.28	47	47.1

The second order system metrics were computed for the square wave trials. It was found not possible to calculate the rise time or the settling time for these trials. Table IV displays the percent overshoot for the proportional PID square wave trial.

TABLE IV
PROPORTIONAL PID SQUARE SECOND ORDER SYSTEM METRICS

K _p	Percent Overshoot (%)
0.75	65.1515
2.0	75.5045
5.0	88.5859

Manually Tuned PID

Next, the PID controller was manually tuned and both a sinusoidal and square wave response was recorded. The manually tuned values for the proportional, integral and derivative gain constants were: 30, 20, 3.

Table V shows the mean, standard deviation, and root mean square of the command and error signals for both the square and sinusoidal manually tuned PID trials.

TABLE V
MANUAL PID SIGNAL STATISTICS

Sinusoidal /Square	Command /Error	Mean	Standard Deviation	RMS
Square	Command	0.1005	99.006	99
Sinusoidal	Command	62.9262	30.488	69.921
Square	Error	0.2485	45.3	45.33
Sinusoidal	Error	0.8542	5.14	5.213

For the square wave, second order system metrics were analyzed. Table VI shows the rise time, percent overshoot, and settling time calculated for the square wave manually tuned PID trials.

TABLE VI
MANUAL PID SQUARE SECOND ORDER SYSTEM METRICS

Rise Time (s)	Percent Overshoot (%)	Settling Time (s)
0.2804	98.35668	0.5985

Ackermann PID

Next, the PID gain constants were calculated using the Ackermann method. The sinusoidal and square responses were recorded, and the calculated proportional and derivative gain constants were: 9.91 and -0.17.

Table VII shows the mean, standard deviation, and root mean square of the command and error signals for both the square and sinusoidal Ackermann calculated PID trials.

TABLE VII
ACKERMANN PID SIGNAL STATISTICS

Sinusoidal /Square	Command /Error	Mean	Standard Deviation	RMS
Square	Command	0.3006	99.005	99
Sinusoidal	Command	63.061	30.471	70.03
Square	Error	-0.7945	47.988	47.99
Sinusoidal	Error	2.812	15.834	16.08

For the square wave, second order system metrics were analyzed. Table VIII shows the rise time, percent overshoot, and settling time calculated for the square wave Ackermann calculated PID trials.

TABLE VIII
ACKERMANN PID SQUARE SECOND ORDER SYSTEM METRICS

Rise Time (s)	Percent Overshoot (%)	Settling Time (s)
0.1558	104.8220	0.2468

IV. DISCUSSION

The goal of this lab was to compare different methods of controlling a system by using a feedback loop. The system was controlled by a bang-bang controller and a PID controller that was tuned in various ways. This lab compared how each controller affected the systems stability, responsiveness, and steady-state accuracy.

The bang-bang controller is a simple controller but can be effective in certain cases. The controller does not have the

ability to scale with the error because it simply just attempts to spin the motor in the direction that would make error go towards zero. For the sinusoidal wave the controller generally performed about the same for all the cases. From Table I it can be seen that all of trials had generally low means and RMS values. Increasing the duty cycle led to generally lower mean but it increased the magnitude of the error as shown by the increased RMS values. Even though the response was faster for the higher duty cycle it had much more oscillation. The largest deadband performed the best for the higher duty cycle but the medium deadband performed the best overall for the smaller duty cycle. The deadband was needed more when the duty cycle was larger since it would over correct more when the controller could move faster.

The square wave bang-bang results shown in Table I further demonstrate the results expected this type of control. The command signal maintained full ± 99 value due to the binary logic of the square wave. The error signal, however, varied differently between different test conditions. The smaller duty cycle had smaller means for the error but the magnitude of the error from RMS was larger compared to the higher duty cycle. Smaller deadbands allowed for faster transitions but larger overshoot and jitter in the response. The highest deadband and duty cycle performed the best in this trial, producing the smallest RMS error. This performance boost implies that the relaxed error tolerance improved stability by reducing unnecessary reversals.

The second-order performance statistics show that the smaller duty cycle had larger percent overshoot, faster rise time, and slower settling time as shown in Table II. This means that the lower duty cycle was good for getting to the target quickly at the cost of accuracy and how long it takes the system to stop jittering.

On the other hand, PID controller are way more complex but significantly harder to implement. This controller can scale its output based on the error it receives. The first method that was tested was just changing the proportional controller. When just using the proportional controller it acts very similarly to the bang-bang controller. The main difference is that it can change the magnitude of the response instead of just being fixed. This means it can give a large response when the error is large and small one when its small. This helps reduce things like overshoot.

Table III shows the command and error statistics for the sinusoidal trials of the proportional controller. One important thing to note is that when K_p was set to 0.75 the system did not move at all. This is because when the error is multiplied by 0.75 to get the input command, that input command when converted to voltage is not strong enough to overcome the static friction of the system. Increase the gain constant will prevent this issue from occurring. When K_p increased to 2.0 the response was underdamped and slow with a large RMS for the error. When K_p increased to 5.0 the RMS for the error drastically decreased but was still relatively large.

For the proportional square wave trials, the system exhibited behavior that deviated from a second order response as shown in Table III. When K_p was 0.75 the motor did not move for the same reason described in the previous paragraph. As the gain increased to 2.0 the system responded more aggressively but the response was slow and highly damped. When the gain was

increased to 5.0 the system responded every more aggressively but still did not show a smoother exponential rise or settling time. The physical interpretation of this is that the system never reaches the 2% error goal for settling time so this can never be computed. The system also does not ever reach the 90% requirement for rise time. This is for the same reason that the low gain constant does not cause movement. Since the command signal output scales with error, once the error is small enough, the voltage sent to the motor will be too small to overcome the friction of the motor and it will stop too early. The percent overshoot shown in Table IV was very large and this was because the response was always damped too quickly.

The manually tuned PID controller performed the best out of all the different configurations. The manually tuned parameters provided a quick response due to the large proportional constant while the derivative term eliminated the oscillations from the overshoot. The integral term allowed for the elimination of the steady state error. Table V shows that for the sinusoidal case, the RMS for the error was very low. The RMS for the square wave was not as low but was lower than almost every other square wave trial. Table VI shows the percent overshoot was very small with a quick rise time and a quick settling time both less than a second.

Finally, the Ackermann tuned PID produced a more conservative and stable response compared to the manually tuned one. The response was faster with a smaller settling time and rise time but had a larger overshoot compared to the manually tuned as shown in Table VIII. This makes sense because the gain constants were mathematically calculated to optimize these values. The settling time is smaller than the desired 0.3s and the percent overshoot is smaller than the desired 5% which means Ackermann's method was successful for tuning the PID controller.

Future Improvement

Future improvements could focus on reducing non-ideal behavior in both the hardware and control implementation. The friction in the motor caused issues with the proportional testing that caused that controller to struggle. Finding a motor that can be used at lower voltages would help reduce this issue. Additionally, measuring the motor voltage rather than the command signal could help provide more accurate input data for the control model. Running an experiment at multiple amplitudes and frequencies can also allow for identification of system non linearities and validate whether the first order approximation used in Lab 1 is still valid under different control conditions.

REFERENCES

- [1] N. Touchton, "Step and Frequency Response of a Motor-Flywheel System," EML 4314C Lab 1 Report, University of Florida, Sept. 2025.
- [2] "EML 4314C Lecture Notes: PID and State-Space Control," unpublished course material.
- [3] "EML 4314C – Fall 2025 Lab 2 Assignment," University of Florida, 2025.